Layer10 Project

Grounded Long Term Memory via Structured Extraction, Deduplication and a Context Graph

Overview:

Modern businesses send and receive a lot of unstructured messages through email, chat rooms, issue trackers, and shared documents. As time goes on, useful organizational knowledge gets spread out across these systems, making it hard to get accurate information about past choices, relationships, and events.

This project creates a long-term memory system that turns unstructured communication into a structured memory graph that can be searched. The system takes entities and claims from raw communication artifacts, removes duplicate information, and saves the resulting knowledge graph with clear evidence references. Every memory item is based on source evidence, which means that you can always find out where the information came from.

The pipeline processes unstructured communication data and produces:

- A structured memory graph consisting of entities, claims, and evidence
- Deduplicated representations of people and relationships
- A retrieval system that returns grounded context packs
- An interactive visualization of the resulting knowledge graph

The goal of the system is not only to extract information but to ensure long-term correctness, meaning the stored memory remains auditable, explainable, and adaptable as source information evolves.

Corpus Selection:

This project uses the Enron Email Dataset, a widely used public corpus containing approximately 500,000 corporate emails from Enron employees.

Dataset source:
Enron Email Dataset

The dataset was chosen because it contains:

- Real-world corporate communication
- Email threads and forwarded messages
- Multiple mentions of the same people and organizations
- Decision-making discussions

These properties create realistic challenges in:

- entity resolution
- duplicate detection
- relationship extraction

The dataset can be downloaded from the CMU archive or Kaggle mirror. After downloading, the ingestion script parses email files and converts them into structured artifact records stored in the system database.

System Architecture:

The system uses a pipeline architecture to change unstructured text into structured memory objects over time. At the start of the pipeline, raw email files are broken down into artifacts that hold the cleaned message text and any other important information. Then, these artifacts go through an extraction stage, where a language model finds entities and relationships that are mentioned in the message. The model gives back a structured JSON representation that shows the entities it found, the relationships between them, and snippets of evidence that support them.

After extraction, a validation stage checks that the returned structure matches the expected schema and that each claim has enough proof. This step stops malformed outputs from getting into the system and lets the pipeline throw away or try again incomplete extractions. After being checked, entities are added to the entity store, and claims that show how entities are related are stored as edges in the memory graph.

Evidence records keep track of the original artifacts, so the system can always find the source text for every stored relationship.

The last step in the pipeline makes a memory graph that can be queried and supports retrieval operations. People can ask the system questions in everyday language, which are then turned into embeddings and compared to stored claim representations. The system gives back a list of ranked results that include entities, relationships, and evidence excerpts that back up those claims. This retrieval process makes sure that

answers are based on original communication artifacts instead of summaries that haven't been checked.

Raw communication artifacts are progressively transformed into structured knowledge through the following steps:

Raw Emails - Email Parsing and Artifact Creation - Structured Extraction (LLM) - Validation & Normalisation - Entity Deduplication - Memory Graph Construction - Retrieval & Visualisation

Ontology and Data Schema:

The memory graph is based on a simple but flexible ontology that includes artifacts, entities, claims, and evidence. Artifacts are the original sources of communication, like emails, and they are the official records from which all knowledge is taken. Entities are real-world things that are talked about in the artifacts, like people, groups, or projects. The system can tell the difference between different types of entities by storing each one with a canonical name and an optional type classification.

Claims are the edges of the memory graph and show how different things are related to each other. A predicate that describes how the two entities interact is used in each claim to show a directional relationship between a subject entity and an object entity. A claim could say that one person sent an email to another, talked about a certain subject, or went to a meeting. Claims also have a confidence score and optional validity timestamps that let the system show both past and present relationships.

Evidence records link claims to the artifacts that back them up. Each evidence entry has a part of the original message and metadata that shows where in the artifact the part is. By requiring that every claim include an evidence pointer, the system ensures that stored memory remains fully grounded and auditable.

Structured Extraction:

The extraction stage uses a language model to turn unstructured email text into structured information. A well-thought-out prompt tells the model to find the entities mentioned in the message, figure out how those entities are related, and pull out a short piece of evidence that backs up each claim.

The model sends back the results in a structured JSON format that can be turned directly into entity and claim objects. The system limits the extraction process to a small schema to keep it consistent and reliable, even though language models can make rich semantic interpretations. This structured approach lets the system keep a consistent ontology while still taking advantage of how language models can understand how people talk to each other in everyday situations.

Deduplication and Canonicalisation:

A major problem in making a memory system is stopping entities and claims from being repeated. Without deduplication, the same person or relationship may show up in different ways, which breaks up the memory graph and makes it harder to find things. To fix this, the system does deduplication at different levels.

Entity canonicalization makes sure that different text versions of the same entity are combined into one canonical record. For instance, "Jeff Skilling," "Jeffrey Skilling," and "J. Skilling" can all be used to talk about the same person. Fuzzy string similarity is used by the system to compare entity names. If two entities are too similar, they are combined. A merge log keeps track of all merge operations so that the process can be checked and undone.

Claim deduplication prevents repeated statements of the same relationship from creating multiple redundant edges in the graph. Instead, repeated claims are merged while preserving multiple evidence references that support the relationship. This approach maintains the strength of evidence without duplicating structural information in the graph.

Memory Graph Design:

The memory graph stores knowledge as a network of entities connected by claims and supported by evidence. In this structure, entities act as nodes while claims represent directed edges between nodes. Evidence records link each claim back to the artifact from which it was derived, forming a traceable chain between stored knowledge and its source communication. The graph structure allows the system to support a wide range of queries, including relationship discovery, semantic search, and exploration of communication networks. Temporal attributes associated with claims allow the graph to represent how relationships change over time, enabling the system to distinguish between historical and current information.

Retrieval and Grounding:

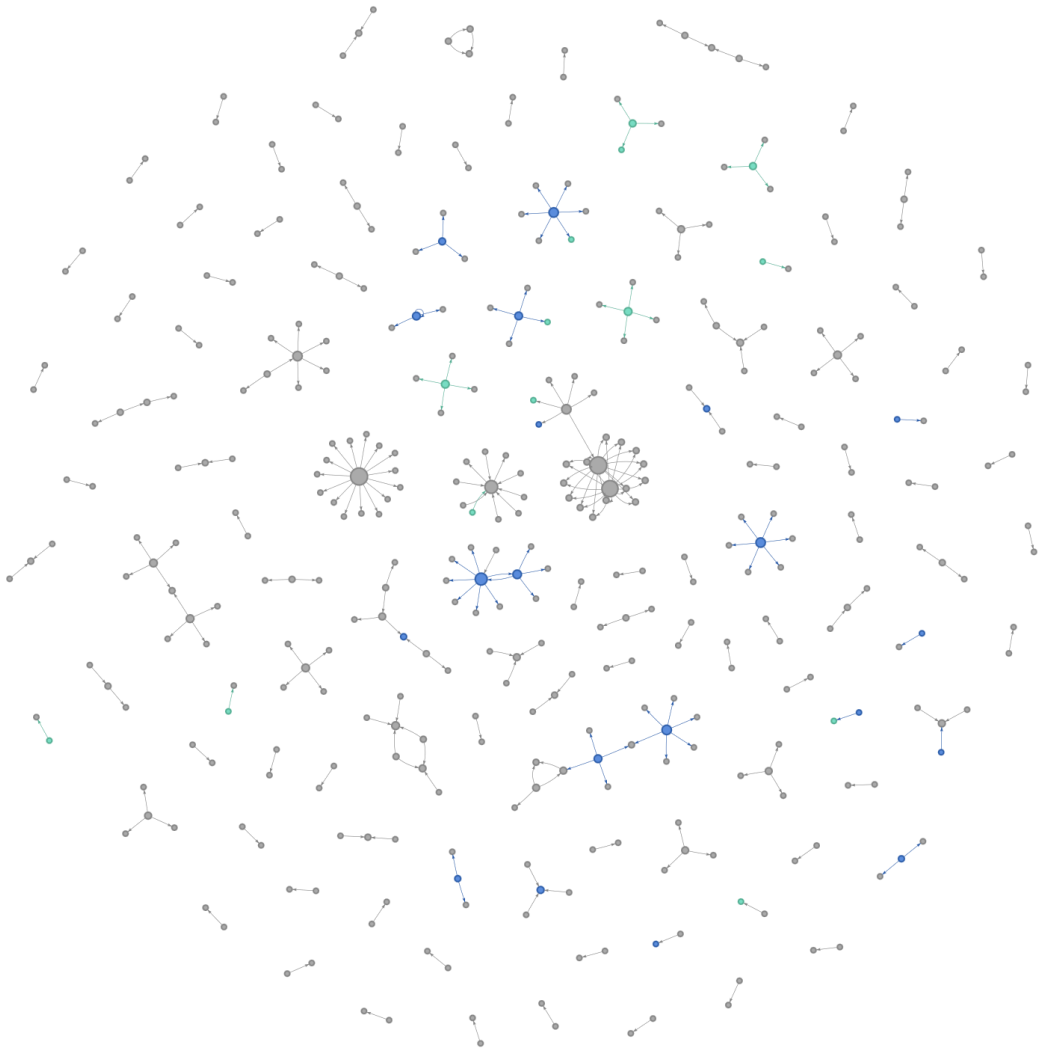
A semantic search mechanism maps user questions to relevant claims in the memory graph to get the information back. When a query is made, the system makes an embedding representation of the query and compares it to the embeddings of stored claims. The claims that are most like each other are returned with their related entities and evidence excerpts.

This method makes sure that the results of the retrieval are based on real communication artifacts. The system doesn't give direct answers; instead, it gives a context pack with structured claims and supporting evidence. This design puts explainability and traceability first, so users can check each piece of information they get by looking at the source excerpt.

Visualisation and Observation:

The system has an interactive visualization that shows entities as nodes and relationships as edges to make it easier to explore the memory graph. The visualization makes central entities stand out by making nodes bigger based on how connected they are. It also uses labels to show what kind of relationship each edge represents. Users can move around the graph, look at how people are connected, and see excerpts of evidence that support certain claims.

This picture helps turn the knowledge that was taken out into an easy-to-understand picture of how people in an organization communicate. The system gives users a clear view of how entities interact and how claims are backed up by evidence by letting them look at the graph visually.



The figure shows an example of a visualization that the system made after running the extraction pipeline on a small part of the Enron email dataset. The graph shows several groups of entities that are linked by relationships that were found. Some nodes look like small pairs, where only one relationship was taken from an email. Others look like bigger groups, showing people who show up a lot in different communications. These hub-like nodes stand for things that are involved in more than one extracted claim, which suggests that there are repeated interactions in the dataset.

The visualization shows how the pipeline turns unstructured email text into a structured network of relationships. This lets users see communication patterns and make sure that the claims they extract create coherent structures instead of random or noisy outputs.

Nodes that appear in many claims become visually central in the graph because they have a higher degree of connectivity. This naturally highlights individuals who participate in many communications or discussions within the dataset. Edge labels represent the predicate extracted from the email text, such as “emailed,” “discussed,” or other relationship indicators identified by the extraction model. Evidence excerpts supporting the claim can also be displayed when inspecting relationships, allowing the user to trace the extracted information back to the original source artifact.

Due to the computational constraints of running local language models, the full Enron dataset could not be processed on the available hardware. The extraction pipeline was therefore executed on a limited subset of the dataset using a processing limit of **50 emails**. Each email requires multiple model calls for entity extraction, relationship identification, and embedding generation, which makes the pipeline computationally expensive when running locally. Running the extraction on 50 emails already placed significant load on the system, and larger runs caused the machine to overheat and shut down during earlier attempts.

Despite this limitation, the resulting graph successfully demonstrates the system’s ability to construct a structured memory representation from unstructured communication. Even with a relatively small sample of emails, the pipeline produced a network containing multiple clusters of entities connected by extracted claims. The visualisation shows both small relationship pairs and larger hub-like structures where certain individuals connect to many others, indicating repeated communication patterns. These clusters illustrate how the system can identify recurring actors and relationships within an organisation’s communication network.

The resulting visualization therefore serves both as an exploration tool and as a validation mechanism for the extraction pipeline. By inspecting the graph, it becomes possible to verify that the extracted relationships form coherent structures rather than isolated or noisy outputs. The graph representation also demonstrates how the system could scale to represent a much larger organisational knowledge network if run on more powerful hardware or distributed infrastructure

Adapting the System to Layer10's Target Environment:

Although this prototype was built using the Enron email dataset, the architecture was intentionally designed to extend to Layer10's target environment, which includes email systems, chat platforms such as Slack or Microsoft Teams, document repositories, and structured issue tracking systems such as Jira or Linear. Supporting these sources would primarily require extending the ontology, modifying the extraction contract, and strengthening grounding and permission mechanisms.

In the current prototype, artifacts correspond to emails, entities represent individuals or organizations, and claims represent relationships extracted from communication. In a production Layer10 environment, the ontology would be expanded to include additional entity types such as teams, channels, documents, tickets, repositories, and components. Claims would also include richer relationship types such as assigned_to, resolved_by, mentioned_in, discussed_in_thread, or blocks, enabling the graph to capture interactions between people, discussions, and structured project artifacts.

For chat platforms such as Slack or Teams, messages would be grouped into conversation threads that serve as artifacts similar to email threads. Individual messages within the thread could act as evidence supporting extracted claims. For structured systems like Jira or Linear, the ingestion process would combine structured metadata (issue IDs, assignees, status changes) with unstructured comment text. This allows the system to fuse structured project data with conversational discussions, linking decisions discussed in chat or email directly to the corresponding tickets, components, or milestones in the memory graph.

Another important design consideration is distinguishing between durable long-term memory and ephemeral conversational context. Not every message or extracted relationship should become a persistent memory. Durable memory items may include decisions, ownership changes, task assignments, or cross-team dependencies that remain relevant over time. Temporary discussions or speculative statements may remain ephemeral unless reinforced by multiple pieces of evidence or later confirmed by structured events such as issue updates. Confidence scores, evidence aggregation, and time-based decay mechanisms can help prevent outdated or speculative information from becoming permanent memory.

Grounding and safety are also critical for long-term organisational memory. Every stored claim should maintain explicit provenance, including the artifact identifier, timestamp, and evidence excerpt from which the claim was derived. If the underlying source message is deleted, edited, or redacted, the corresponding claims should be invalidated

or updated. Maintaining strong grounding ensures that retrieved knowledge remains verifiable and auditable over time.

Permission control is another key requirement in real organisational environments. Memory retrieval must respect the access permissions of the underlying data sources. A user querying the system should only receive claims grounded in artifacts they are authorized to access. This can be implemented by attaching access control metadata to artifacts and filtering retrieval results based on the requesting user's permissions.

Finally, operational considerations become important when scaling the system to large organizations. The pipeline should support incremental ingestion as new messages, tickets, or documents are created. Instead of reprocessing the entire corpus, the system would process only new or updated artifacts and update the graph accordingly. Evaluation and regression testing are also important to ensure extraction quality does not degrade when models or prompts are updated. Monitoring metrics such as extraction error rates, duplicate entity frequency, and claim confidence distributions would help maintain the quality of the long-term memory system as it evolves.

Conclusion:

This project shows how a long-term memory system can be built from scattered communication data. The system makes a clear and searchable knowledge graph from unstructured communication by combining language model extraction with careful schema design, deduplication strategies, and explicit evidence grounding. The memory graph that comes out of this process makes it easy to find organisational knowledge while keeping clear links to the original source artifacts.

The method shows how important grounding, traceability, and deduplication are for making reliable memory systems. These principles make sure that stored knowledge can be checked over time and changed to fit new situations in the organization. As communication systems keep making more and more data, systems like this one help organisations keep their long-term memory organized and easy to check.